LLM Judge Bias and Sensitivity

Inference Pipeline Architecture Document

Swiss AI Stack · Qwen3-30B-A3B-Instruct-2507

ETH Zürich – Training Better Preference Models

March 2026

Contents

1 Overall Architecture

Design Philosophy

The pipeline is a **pure HTTP client** that sends judge queries to the **Swiss AI Stack** (https://serving.swissai.svc.cscs.ch/v1). There is no local cluster, no Slurm scheduler, and no local vLLM instance. All inference is performed remotely via OpenAI-compatible REST requests; the local code handles only data loading, prompt construction, async dispatch, response parsing, and metric computation.

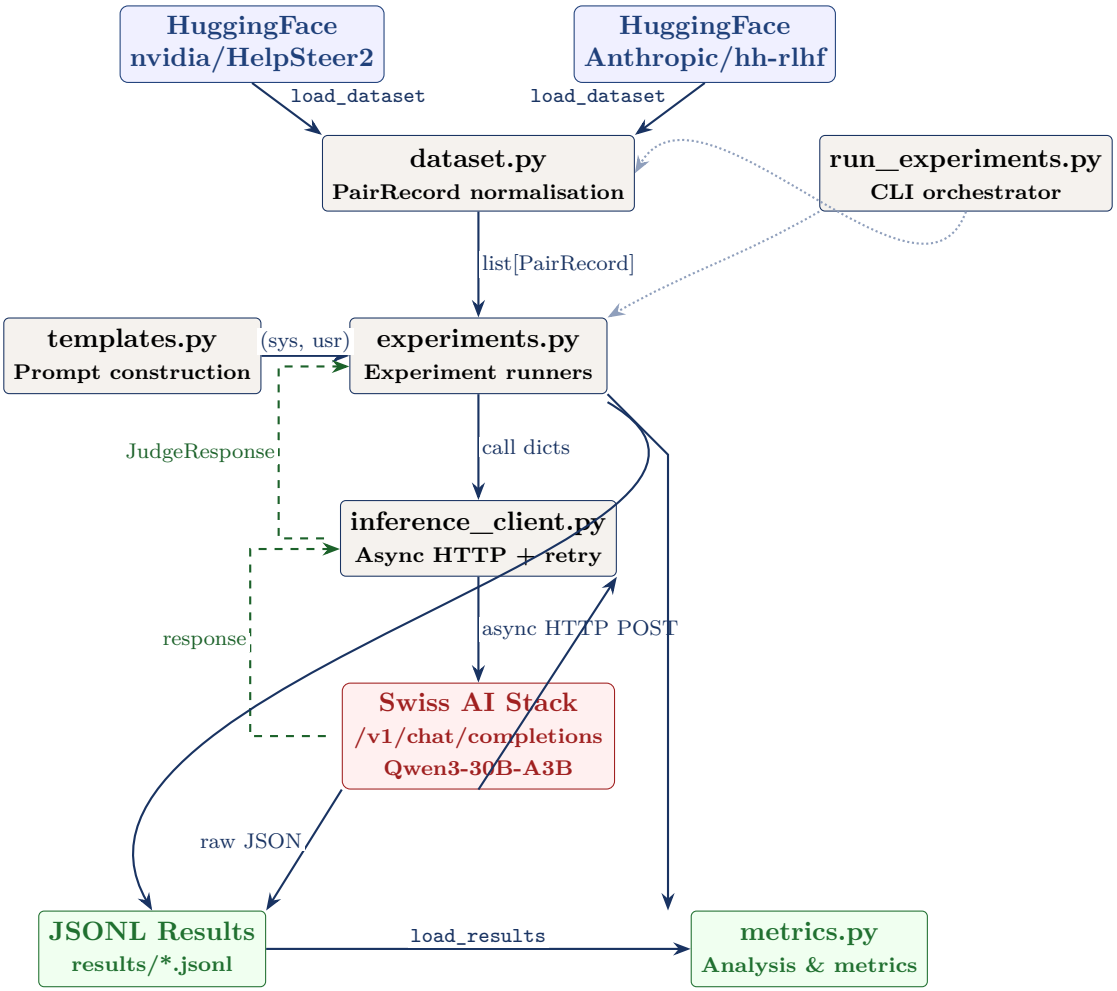
Key Terminology

System prompt A hidden instruction sent to the model *before* the user's message. It sets the model's role and behaviour — for example, "You are an expert rater." The user never sees this text; it tells the model *how* to act. In this pipeline, the system prompt defines the judge persona and output format.

User prompt The visible message that the model responds to. It contains the actual content to evaluate: the original user question and the two candidate responses (A and B). The model reads both the system prompt and the user prompt, but only the user prompt carries the data.

Every API call in this pipeline sends exactly one system prompt and one user prompt. The system prompt varies across experiments (different templates), while the user prompt varies across dataset pairs (different questions and responses).

1.1 Data-Flow Diagram



Data-Flow Summary

1. **dataset.py** loads from HuggingFace (HelpSteer2 or hh-rlhf), normalises each sample to a `PairRecord` (prompt_id, prompt, response_a, response_b, gold_label).
2. **experiments.py** builds judge call lists from pairs and template configurations, dispatches them via `SwissAIClient.batch_judge`. For example, the position bias experiment creates two calls per pair — one in the original A/B order and one flipped — producing a list of dictionaries like `{system_prompt: ..., user_prompt: ..., prompt_id: "a3f8", condition: "AB"}`.
3. **inference_client.py** sends async HTTP POST requests to the Swiss AI Stack's OpenAI-compatible `/v1/chat/completions` endpoint.
4. Responses are parsed using pattern matching (the code searches the model's output for verdict lines like "Verdict: A" or a standalone A/B/C at the end of the text), saved to JSONL, and fed to **metrics.py** for analysis.

2 Module Responsibilities

2.1 inference_client.py

Async HTTP Client for the Swiss AI Stack

Wraps the remote OpenAI-compatible endpoint with structured request/response handling, concurrency control, and robust retry logic.

Key Components

`InferenceConfig`

Dataclass holding all configuration:

- `base_url`: https://serving.swissai.svc.cscs.ch/v1
- `api_key`: Bearer token (from `SWISSAI_API_KEY` env var)
- `model`: `Qwen/Qwen3-30B-A3B-Instruct-2507`
- Generation params: `max_tokens=2048`, `temperature=0.0`, `top_p=1.0`
- Retry config: `max_retries=5`, `retry_base_delay=2.0s`, `retry_max_delay=60.0s`
- Concurrency: `concurrent_requests=8`

`JudgeResponse`

Dataclass for structured results: `prompt_id`, `experiment_id`, `condition`, `raw_text`, `label` (A/B/C/null), `thinking` (extracted CoT or null), `latency_s`, `total_tokens`, `parse_ok`, `error`.

Label Extraction

Three-tier pattern-matching cascade (the code searches the model's output text for specific formats, trying each rule in order until one matches):

1. **Verdict lines**: looks for explicit markers like "Verdict: B", "Answer: A", "Output: C"
2. **Bare label**: a single A, B, or C standing alone at the end of the text
3. **Fallback**: scans the entire text and picks the last standalone uppercase A, B, or C found

If the model produced a `<think>...</think>` block (internal reasoning), that block is stripped before label extraction and stored separately in the result.

`SwissAIClient`

Async context manager creating an `aiohttp.ClientSession` with 300 s timeout. Uses `asyncio.Semaphore` for concurrency limiting. The `judge()` method sends a single judge call; `batch_judge()` dispatches a list of calls via `asyncio.gather`.

2.2 templates.py

Judge Prompt Templates

Defines all system-prompt variants, criteria, and the shared user-prompt builder. Each template is registered in the `TEMPLATES` dictionary.

System Prompt Variants

ID	Framing	Used In
`expert_rater`	Baseline expert rater	B1, B2, B3, B4
`llm_judge`	"LLM used to judge"	B2
`neutral`	Imperative, no persona	B2
`academic`	Formal quality evaluator	B2
`minimal`	One-liner	B2
`reason_then_judge`	Explain reasoning, then verdict	B3
`structured_reasoning`	Rate on criteria, then verdict	B3
`expert_rater_alt1`	"superior response"	B4
`expert_rater_alt2`	"AI-generated answers / preferable"	B4
`expert_rater_alt3`	"serves the user"	B4

Criteria Variants

Five criteria defined in the `CRITERIA` dict: `helpful`, `quality`, `accurate`, `harmless`, `concise`. Each maps to a natural-language fragment (e.g. "more helpful to the user").

`build_prompt()`

Resolves a `template_id` + `criterion` into a `(system_prompt, user_prompt)` tuple. The user prompt follows a fixed structure:

User Prompt Format

```
[User Prompt]
{prompt}

[Response A]
{response_a}

[Response B]
{response_b}

Which response is {criterion}?
```

2.3 dataset.py

Dataset Loading and Normalisation

Loads HuggingFace preference datasets and normalises them into a unified `PairRecord` format suitable for pairwise judge evaluation.

`PairRecord` Dataclass

`prompt_id`
: Unique identifier (MD5 hash or index-based)

`prompt` The original user query

`response_a`
: First response (chosen / higher-rated in original data)

`response_b`
: Second response (rejected / lower-rated)

`gold_label`
: "A", "B", or "C" derived from human preference

The `flipped()` method returns a new record with A/B swapped and the gold label adjusted (`A`↔`B`, `C` unchanged).

Dataset Loaders

HelpSteer2 (`nvidia/HelpSteer2`)
A scored (not pairwise) dataset. Pairs are constructed by grouping responses by prompt and taking the highest-scored vs. lowest-scored response. Score = mean(helpfulness + correctness + coherence) / 3.

HH-RLHF (`Anthropic/hh-rlhf`)
A pairwise dataset with chosen/rejected columns. The loader extracts the last assistant turn from each multi-turn conversation. Gold label is always "A" (chosen is preferred).

2.4 experiments.py

Experiment Runners (B1–B4)

Four async functions, each building call dicts, dispatching via `batch_judge`, and saving JSONL results. Contains condition definitions for each experiment.

Function	ID	Conditions
`run_position_bias`	B1	"AB" (original), "BA" (flipped)
`run_template_sensitivity`	B2	5 templates: expert_rater, llm_judge, neutral, academic, minimal
`run_reasoning_depth`	B3	3 conditions: no_reasoning, reason_then_judge, structured_reasoning
`run_input_sensitivity`	B4	4 templates × 5 criteria = 20 conditions

How Call Lists Are Built — Worked Example

Each experiment loops over pairs (and conditions) to build a flat list of dictionaries. Each dictionary contains everything needed for one API call. Here is a simplified example for position bias (B1) with 2 pairs:

B1 Call List Construction (Pseudocode)

```
pairs = [pair_1, pair_2]   # loaded from dataset

calls = []
for pair in pairs:
    # Original order: Response A first, Response B second
    sys_prompt, usr_prompt = build_prompt("expert_rater",
        pair.prompt, pair.response_a, pair.response_b)
    calls.append({
        "system_prompt": sys_prompt,
        "user_prompt":   usr_prompt,
        "prompt_id":     pair.prompt_id,
        "condition":     "AB"
    })

    # Flipped order: Response B first, Response A second
    flipped = pair.flipped()
    sys_prompt, usr_prompt = build_prompt("expert_rater",
        flipped.prompt, flipped.response_a, flipped.response_b)
    calls.append({
        "system_prompt": sys_prompt,
        "user_prompt":   usr_prompt,
        "prompt_id":     pair.prompt_id,
        "condition":     "BA"
    })

# Result: 4 calls (2 pairs x 2 orders)
# All dispatched at once via client.batch_judge(calls)
```

For B2, the outer loop iterates over 5 templates instead of 2 orders. For B3, it iterates over 3 reasoning conditions. For B4, it iterates over 4 template variants $\times$ 5 criteria = 20 combinations.

2.5 metrics.py

Metric Computation and Reporting

Loads JSONL results and computes experiment-specific metrics. Provides `print_summary()` for console output.

`compute_position_bias`
: Position consistency, bias rates, positional preference breakdown.

`compute_pairwise_agreement`
: Overall pairwise agreement, per-pair agreement rate, label distribution per condition, most volatile pairs. Used for both B2 and B4.

`compute_thinking_accuracy`
: Accuracy vs. gold labels per condition, agreement vs. no_reasoning baseline, average latency per condition. Used for B3.

2.6 run_experiments.py

CLI Orchestrator

Top-level entry point with `argparse`. Loads dataset, configures the inference client, runs selected experiments, computes metrics, and prints summaries.

Example Invocations

```
# Run all experiments (200 pairs, default settings)
python3 run_experiments.py

# Quick smoke test: position bias only, 20 pairs
python3 run_experiments.py --experiments position_bias --n-pairs 20

# Run only reasoning depth experiment
python3 run_experiments.py --experiments reasoning_depth

# Full configuration
python3 run_experiments.py --dataset nvidia/HelpSteer2 \
    --split validation --n-pairs 200 --criterion helpful
```

CLI flags include: `-experiments`, `-n-pairs`, `-dataset`, `-split`, `-criterion`, `-template`, `-output-dir`, `-model`, `-base-url`, `-concurrency`, `-seed`.

3 The Four Experiments

3.1 B1: Position Bias

B1 – Position Bias Detection

Research question: Does the LLM judge prefer whichever response appears in a particular position (first or second), regardless of content?

Method: For each pair, judge both orders:

- **AB** (original): Response A first, Response B second
- **BA** (flipped): Response B first, Response A second

A *consistent* judge should prefer the same underlying response in both orderings. If the AB verdict is "A", the BA verdict should be "B" (the same response, now in position B).

Metrics:

`position_consistency` Percentage of pairs where the flipped label matches the expected inversion.

`position_bias_rate` 1 − consistency; the fraction where the model's verdict changed with position.

`bias_toward_first_position` Both AB and BA say "A" (always picks first).

`bias_toward_second_position` Both AB and BA say "B" (always picks second).

3.2 B2: Template Sensitivity

B2 – Template Sensitivity Analysis

Research question: How sensitive is the judge's verdict to the framing of the system prompt, given identical data and response ordering?

Method: Judge each pair with 5 different system prompt templates:

Template	Character
`expert_rater`	"You are an expert rater..."
`llm_judge`	"You are an LLM used to judge..."
`neutral`	"Compare the following two responses..."
`academic`	"You are a quality evaluator..."
`minimal`	"Pick the better response: A, B, or C (tie)."

Same data, same order, same criterion — only the system prompt changes.

Metrics:

`overall_pairwise_agreement` Average pairwise agreement across all template pairs for each prompt.

`label_distribution_per_condition` A/B/C breakdown for each template.

`most_volatile_pairs` The 10 pairs with lowest agreement across templates.

3.3 B3: Prompt-Elicited Reasoning

B3 – Reasoning Depth

Research question: Does explicitly asking the model to reason before giving its verdict improve accuracy against human gold labels?

All reasoning in this experiment is **prompt-elicited**: the prompt itself tells the model how much (or how little) to reason. There is no native API thinking-budget feature involved — the only thing that changes between conditions is the system prompt template.

Conditions:

Condition	What the prompt tells the model
`no_reasoning`	Just output a single letter: A, B, or C. No explanation requested. Uses the baseline `expert_rater` template.
`reason_then_judge`	"Explain your reasoning about the strengths and weaknesses of each response, then provide your final verdict." The model argues its decision before answering.
`structured_reasoning`	"Rate each response on helpfulness, accuracy, and coherence, then provide your final verdict." The model evaluates on specific criteria before deciding.

Metrics:

`accuracy_vs_gold` Agreement with human preference labels per condition.
`delta_accuracy` Improvement of each reasoning condition over **`no_reasoning`**.
`agreement_vs_no_reasoning` Fraction of verdicts matching the no-reasoning baseline.
`avg_latency_s` Average response time per condition (reasoning conditions produce longer outputs and take more time).

B3 Design Note

This experiment tests **prompt-elicited Chain of Thought** — the model is explicitly told in the prompt to argue its decision before answering. This is fundamentally different from a native API thinking-budget feature (like Anthropic's extended thinking or Qwen3's `enable_thinking` parameter). Here, the reasoning appears directly in the model's visible output because the *prompt* requested it, not because a special API mode was activated.

3.4 B4: Input Sensitivity

B4 – Input / Wording Sensitivity

Research question: How sensitive is the judge to minor paraphrasing of the system prompt and changes in evaluation criterion?

Method: Cross 4 minor wording variants of the expert_rater template with 5 evaluation criteria:

Template Variant	Key Wording Difference
`expert_rater`	Baseline: "determine which response is better"
`expert_rater_alt1`	"identify the superior response"
`expert_rater_alt2`	"AI-generated answers" + "which answer is preferable"
`expert_rater_alt3`	"Evaluate both…select the one that better serves the user"

Criterion	Phrase
`helpful`	"more helpful to the user"
`quality`	"of higher overall quality"
`accurate`	"more accurate and factually correct"
`harmless`	"safer and more appropriate"
`concise`	"more concise while still fully answering the question"

Total: $4 \times 5 = 20$ conditions per pair.

Metrics:

`pairwise_agreement` Agreement rate across all 20 conditions.

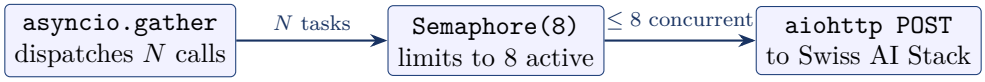
`criterion_drift` How much the label changes when only the criterion changes (template held fixed).

4 Async Concurrency and Retry/Backoff Logic

Concurrency Model

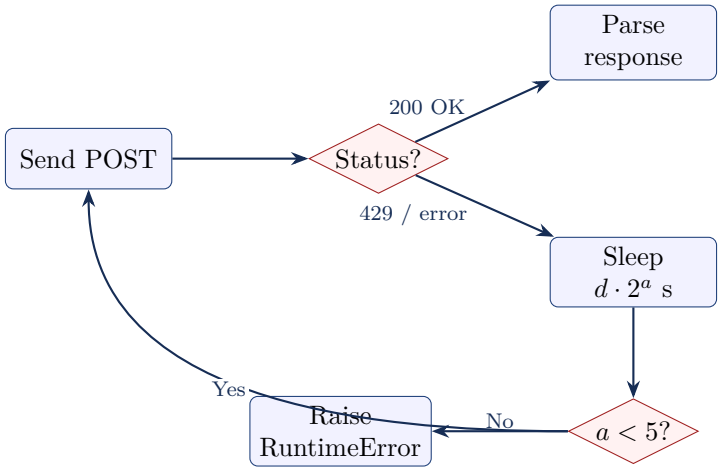
The pipeline uses Python's `asyncio` event loop with `aiohttp` for non-blocking HTTP. All experiment calls are dispatched simultaneously via `asyncio.gather`, bounded by a semaphore.

4.1 Concurrency Control



- `SwissAIClient` creates an `aiohttp.ClientSession` as an async context manager with a 300 s total timeout per request.
- An `asyncio.Semaphore(concurrent_requests)` (default 8) ensures no more than 8 HTTP requests are in flight simultaneously.
- `batch_judge(calls)` wraps all calls in `asyncio.gather` for maximum parallelism within the semaphore bound.

4.2 Exponential Backoff Retry



- On HTTP 429 (rate limit) or `aiohttp.ClientError`, the client waits min(`base_delay` $\times$ $2^{\texttt{attempt}}$, `retry_max_delay`) seconds.
- Default: `base_delay` = 2.0 s, `max_delay` = 60.0 s, so delays are 2, 4, 8, 16, 32 s (capped at 60 s).
- After 5 failed attempts, a `RuntimeError` is raised.
- Successful responses (HTTP 200) are parsed immediately; the label extractor runs on the response text.

5 Output JSONL Format

Result Serialisation

Each experiment writes one JSONL file to the `results/` directory. Each line is a single JSON object representing one judge call.

JSONL Record Schema

```
{
  "prompt_id":     "a3f8b2c901",
  "experiment_id": "position_bias",
  "condition":     "AB",
  "raw_text":      "Response A is more helpful because...\nVerdict: A",
  "label":         "A",
  "thinking":      null,
  "latency_s":     2.341,
  "total_tokens":  487,
  "parse_ok":      true,
  "error":         null
}
```

Field	Type	Description
`prompt_id`	`str`	Unique identifier for the dataset pair (MD5 hash or index-based).
`experiment_id`	`str`	Experiment name: `position_bias`, `template_sensitivity`, `reasoning_depth`, or `input_sensitivity`.
`condition`	`str`	Experimental condition (e.g. "AB", "BA", template name, "reason_then_judge", "expert_rater_alt2_accurate").
`raw_text`	`str`	Full model output text, unmodified.
`label`	`str|null`	Extracted verdict: "A", "B", "C", or `null` if parsing failed.
`thinking`	`str|null`	Content of `<think>...</think>` block if present; `null` otherwise.
`latency_s`	`float`	Wall-clock time for this request in seconds.
`total_tokens`	`int`	Total tokens (prompt + completion) as reported by the API.
`parse_ok`	`bool`	Whether label extraction succeeded.
`error`	`str|null`	Error message if the request failed; `null` on success.

Results are loaded by `metrics.py`'s `load_results(path)` function, which reads each line as a JSON object and returns a `list[dict]` for downstream analysis.

File Layout

Results Directory Structure

```
results/
  position_bias/
    expert_rater_helpful.jsonl
  template_sensitivity/
    criterion_helpful.jsonl
  reasoning_depth/
    criterion_helpful.jsonl
  input_sensitivity/
    all.jsonl
```

6 Infrastructure Summary

No Local Compute

This pipeline performs **zero local GPU inference**. All model calls are HTTP requests to the Swiss AI Stack. The local machine only needs Python 3.10+ with `aiohttp`, `datasets`, and `numpy`.

Component	Detail
Inference backend	Swiss AI Stack (CSCS)
Endpoint URL	`https://serving.swissai.svc.cscs.ch/v1`
API compatibility	OpenAI `/v1/chat/completions`
Model	`Qwen/Qwen3-30B-A3B-Instruct-2507`
Authentication	Bearer token via `SWISSAI_API_KEY`
Protocol	HTTPS + JSON
Client library	`aiohttp` (async)
Concurrency	Semaphore-bounded (default 8)
Retry strategy	Exponential backoff, max 5 retries
Local dependencies	Python 3.10+, aiohttp, datasets, numpy
Scheduler / cluster	**None** (no Slurm, no local vLLM)

End of Architecture Document